Health Insurance Claims Simulator

Project summary | MITRAz Skills BRD v1.0 (16 Oct 2026) | Individual full-stack student project

1. What is it?

A **fictional** web app where a patient or hospital submits a health-insurance claim, claims staff review it, and an admin records a **simulated** payment. It teaches business workflows, role-based permissions, database design and full-stack development. Nothing is real: no real policies, money, medical files, SMS or hospital connections, and all data is synthetic.

2. Tech stack

HTML, CSS, JavaScript, Bootstrap (front end); Python + Flask with Jinja templates (back end); one MySQL database. Werkzeug password hashing; CSRF protection; ORM queries.

3. Four roles

Role	Can do	Must not
Policyholder	View own policies, members, claims; draft/submit reimbursement claims; answer more-info requests	Touch another holder's claim; approve or mark paid
Hospital desk	Draft/submit **cashless** claims for its own network hospital; update checklist	Use another hospital; approve/reject; see unrelated claims
Claims officer	See submitted claims, assign to self, review, request info, approve (with eligible amount + reason) or reject (with reason)	Edit master policies; approve a draft
Administrator	Manage hospitals, plans, user roles; record simulated payment on approved claims; view reports	Change an outcome without a history event

Permissions must be enforced **on the server** (Flask routes and queries), not just by hiding buttons.

4. Claim workflow and states

From	To	Who / condition
draft	submitted	Creator; fields valid; admission not in the future
submitted	in_review	Claims officer assigned to the claim
in_review	more_info	Officer; reason + requested document recorded
more_info	submitted	Original submitter responds
in_review	approved	Officer; 0 < eligible ≤ requested and ≤ remaining cover; reason
in_review	rejected	Officer; written reason
approved	paid	Administrator; amount equals computed payout
rejected / paid	none	Terminal; no reopening

Every state change writes a history event (actor, old state, new state, time, note) in one database transaction. Approval is never automatic.

5. Business rules

• **Payout** = round((eligible – min(eligible, deductible)) × (1 – copay% / 100), 2), using Decimal. Example: Rs. 20,000, deductible Rs. 2,000, co-pay 10% gives Rs. 16,200.

• **Remaining cover** = plan annual limit – earlier paid payouts on that policy in the current policy year.

• **Policy active:** status active and admission date within start/end dates (inclusive). **Treatment** must be in the plan's covered types.

• **Duplicate warning:** same member, hospital, dates and claim type shows a warning; it does not auto-reject.

• **Cashless:** hospital must be a network hospital matching the desk user's hospital; payee is the hospital. **Reimbursement:** any hospital; payee is the policyholder.

• **Member** must belong to the chosen policy (checked server-side).

6. Main features (FR01–FR10)

Login/roles	Policy and member lookup	Claim create + draft	Document checklist
Work queue + assignment	Assessment and decision	Simulated settlement	History timeline + flash messages
Search + filters (with reset)	Dashboards and reports		

Screens: Login, role dashboard, My policies, claim list, claim detail, new/edit draft, review queue, admin, reports. Use Jinja template inheritance, Bootstrap, GET filters and redirect-after-POST.

7. Data model (MySQL)

users, hospitals, plans, policies, members, claims, claim_documents, claim_history, settlements (one per claim). Foreign keys everywhere; unique username; indexes on claims(policy_id,status), claims(officer_user_id,status), claim_history(claim_id,timestamp). Keep models, routes/services, templates, static files and config in separate files.

8. Starter dataset (ZIP)

9 CSVs + README: 5 hospitals (3 network), 3 plans, 15 users (8 holders, 4 hospital desks, 2 officers, 1 admin), 8 policies, 11 members, 13 claims, checklist rows, history, 2 settlements. **Import order:** hospitals → plans → users → policies → members → claims → claim_documents → claim_history → settlements. No password hashes supplied; create them at seeding time. Read CSVs as utf-8-sig (BOM); empty cell = NULL; map user_id/claim_id to your schema.

Claim	Learning case
1001 / 1008	Paid (settlement Rs. 21,600 / Rs. 32,850)
1002	Daycare not covered by Essential Care
1003 / 1009	Approved, awaiting settlement
1004	In review
1005 / 1011	More info; discharge summary missing
1006	Expired policy
1007 / 1012	Submitted, unassigned
1011 + 1012	Potential duplicate
1010 / 1013	Drafts; 1013 has a future admission date

9. Non-functional needs and testing

• **Security:** hashed passwords, secrets outside source control, server-side role and ownership checks, CSRF, escaped output, no sensitive text in logs.

• **Reliability:** transactions with rollback; data persists after restart. **Usability:** clear labels, empty states, responsive, keyboard-friendly.

• **Tests required:** unauthorized URL/POST, invalid transitions, member-policy mismatch, hospital scope, duplicate warning, payout calculation, restart persistence, two-officer assignment conflict.

10. Delivery and demo

Six working slices: (1) ER diagram, schema, seed; (2) auth + dashboards + denied-access demo; (3) claim create/submit + checklist; (4) officer queue, review, decisions; (5) settlement, calculation, filters, reports; (6) testing, README, final demo. **Final demo:** holder drafts and submits; hospital desk creates cashless and another hospital is rejected; officer claims, requests info, approves and shows the calculation; admin records settlement, app restarts and data persists; then show a blocked cross-user view, an invalid transition and combined filters.

Out of scope: real file uploads/OCR, fraud scores, payment gateway, email/SMS, insurer APIs, regulatory SLAs, clinical logic. All rules are classroom inventions, not IRDAI rules or real insurance advice.